

CETAM — Centro de Educação Tecnológica do Amazonas
Curso Técnico de Nível Médio em Informática — Linguagem de Programação III (Java)

Projeto 4 — MonitoraRioNegro

Monitoramento de Níveis de Cheia e Seca

Documentação da API interna e das classes Java

Avaliação AV3 — Apresentação de Projeto (27/07)

Profa. Priscila Gonçalves

Equipe: Antônio Augusto • Chester Amorim • Gabriela Mota • José Roberto

Contextualização Amazônica aplicada ao desenvolvimento orientado a objetos

Visão Geral do Projeto

O MonitoraRioNegro é um protótipo educacional em Java para cadastrar estações de monitoramento, registrar medições diárias do nível do Rio Negro e organizar informações de cheia e seca. O sistema funciona em modo texto, mantém os dados em memória durante a execução e permite consultar histórico, variação, alertas, estatísticas e comparação entre estações.

Entregáveis do projeto

- Apresentação de slides com o nome completo dos quatro integrantes na capa.
- Repositório no GitHub contendo o código-fonte organizado em pacotes.
- Arquivo README.md com descrição, pré-requisitos e instruções de execução.
- Demonstração do funcionamento do sistema e das operações CRUD em console.
- Documentação da API interna das classes, atributos, construtores, métodos e exceções.

Critérios técnicos atendidos

- Mais de quatro classes próprias, com atributos privados e métodos de acesso.
- Interface ClassificadorNivel e polimorfismo com ClassificadorPadrao e ClassificadorPersonalizado.
- Coleções da API Java: ArrayList, LinkedHashMap, HashSet e EnumMap.
- Exceções próprias para nível inválido, medição duplicada e registro não encontrado.
- CRUD completo de estações e medições por meio de menu em modo texto.
- Organização em pacotes: model, classificacao, service, exception, util e main.

Observação: as faixas de classificação e os dados demonstrativos do sistema não são informações oficiais.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

Contexto Amazônico	O projeto considera os ciclos de cheia e seca do Rio Negro e os impactos que essas variações podem causar na vida ribeirinha, na navegação, no transporte e no acesso a comunidades. O sistema organiza dados de estações e medições para fins acadêmicos.
Objetivo Geral	Desenvolver um sistema Java que registre medições diárias, classifique automaticamente o estado do nível, gere alertas, apresente histórico e variação e compare dados entre duas ou mais estações.

Requisitos Funcionais

- Cadastrar, listar, atualizar e remover estações de monitoramento.
- Cadastrar, listar, atualizar e remover medições diárias em metros.
- Classificar cada medição como seca, normal, atenção, cheia ou emergência.
- Gerar alertas hidrológicos quando a classificação não for normal.
- Exibir histórico e variação entre medições consecutivas.
- Comparar a medição mais recente de duas ou mais estações.
- Gerar resumo estatístico e exportar o histórico para arquivo CSV.

Classes e Pacotes Principais

- model: EstacaoMonitoramento, Medicao, AlertaHidrologico e EstadoNivel.
- classificacao: ClassificadorNivel, ClassificadorPadrao e ClassificadorPersonalizado.
- service: EstacaoService, MedicaoService e RelatorioService.
- exception: NivelInvalidoException, MedicaoDuplicadaException e RegistroNaoEncontradoException.
- util: LeitorConsole; main: Main.

Conceitos de POO Demonstrados

- Encapsulamento de atributos e controle das coleções internas.
- Interface e polimorfismo em tempo de execução por estação.
- Enum para os estados hidrológicos.
- Composição entre estação, medições, classificador e alertas.
- Herança das exceções personalizadas a partir de Exception.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

model — entidades de domínio

class EstacaoMonitoramento

Representa uma estação cadastrada e mantém seu classificador e suas medições.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	int	id	Identificador único da estação.
private	String	nome	Nome da estação.
private	String	cidade	Município da estação.
private	String	localizacao	Descrição do local.
private	ClassificadorNivel	classificador	Estratégia de classificação utilizada.
private final	List<Medicao>	medicoes	Medições associadas à estação.

CONSTRUTORES

Assinatura	Descrição
EstacaoMonitoramento(int id, String nome, String cidade, String localizacao, ClassificadorNivel classificador)	Inicializa a estação e cria uma lista vazia de medições.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	int getId()	—	Retorna o identificador.
public	String getNome()	—	Retorna o nome.
public	void setNome(String nome)	—	Atualiza o nome.
public	String getCidade()	—	Retorna a cidade.
public	void setCidade(String cidade)	—	Atualiza a cidade.
public	String getLocalizacao()	—	Retorna a localização.
public	void setLocalizacao(String localizacao)	—	Atualiza a localização.
public	ClassificadorNivel getClassificador()	—	Retorna a estratégia atual.
public	void setClassificador(ClassificadorNivel classificador)	—	Altera a estratégia.
public	List<Medicao> getMedicoes()	—	Retorna uma visão não modificável.
public	void adicionarMedicao(Medicao medicao)	—	Adiciona uma medição.
public	boolean removerMedicao(Medicao medicao)	—	Remove a medição indicada.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

model — entidades de domínio

CLASS Medicao

Representa o registro diário de nível, chuva, temperatura e observação.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	int	id	Identificador único da medição.
private	LocalDate	data	Data da medição.
private	double	nivel	Nível do rio em metros.
private	double	chuva	Chuva registrada em milímetros.
private	double	temperatura	Temperatura em graus Celsius.
private	String	observacao	Informação complementar.

CONSTRUTORES

Assinatura	Descrição
Medicao(int id, LocalDate data, double nivel, double chuva, double temperatura, String observacao)	Cria uma medição com todos os dados informados.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	int getId()	—	Retorna o ID.
public	LocalDate getData()	—	Retorna a data.
public	void setData(LocalDate data)	—	Atualiza a data.
public	double getNivel()	—	Retorna o nível.
public	void setNivel(double nivel)	—	Atualiza o nível.
public	double getChuva()	—	Retorna a chuva.
public	void setChuva(double chuva)	—	Atualiza a chuva.
public	double getTemperatura()	—	Retorna a temperatura.
public	void setTemperatura(double temperatura)	—	Atualiza a temperatura.
public	String getObservacao()	—	Retorna a observação.
public	void setObservacao(String observacao)	—	Atualiza a observação.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

model — entidades de domínio

class AlertaHidrologico

Registro imutável gerado quando uma medição apresenta estado diferente de normal.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	int	id	Identificador do alerta.
private final	int	estacaold	ID da estação relacionada.
private final	String	nomeEstacao	Nome da estação.
private final	LocalDateTime	dataHora	Momento da geração.
private final	EstadoNivel	estado	Estado classificado.
private final	double	nivel	Nível que originou o alerta.
private final	String	mensagem	Recomendação gerada.

CONSTRUTORES

Assinatura	Descrição
AlertaHidrologico(int id, int estacaoId, String nomeEstacao, LocalDateTime dataHora, EstadoNivel estado, double nivel, String mensagem)	Inicializa todos os dados do alerta.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	int getId()	—	Retorna o ID do alerta.
public	int getEstacaold()	—	Retorna o ID da estação.
public	String getNomeEstacao()	—	Retorna o nome da estação.
public	LocalDateTime getDataHora()	—	Retorna data e hora.
public	EstadoNivel getEstado()	—	Retorna o estado.
public	double getNivel()	—	Retorna o nível.
public	String getMensagem()	—	Retorna a mensagem.

enum EstadoNivel

Enumeração dos estados utilizados na classificação das medições.

CONSTANTES

SECA, NORMAL, ATENCAO, CHEIA, EMERGENCIA

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	String getDescricao()	—	Retorna a descrição legível do estado.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

classificacao — interface e estratégias

INTERFACE `ClassificadorNivel`

Contrato usado pelas estratégias de classificação de níveis.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	EstadoNivel classificar(double nivel)	NivelInvalidoException	Classifica o nível informado.
public	String gerarRecomendacao(EstadoNivel estado)	—	Gera orientação conforme o estado.
public	String getDescricaoFaixas()	—	Descreve as faixas configuradas.

`class ClassificadorPadrao implements ClassificadorNivel`

Implementação com faixas fixas demonstrativas.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private static final	double	LIMITE_SECA	Valor 15,0 m.
private static final	double	LIMITE_ATENCAO	Valor 27,0 m.
private static final	double	LIMITE_CHEIA	Valor 28,0 m.
private static final	double	LIMITE_EMERGENCIA	Valor 29,0 m.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	EstadoNivel classificar(double nivel)	NivelInvalidoException	Aplica as faixas padrão.
public	String gerarRecomendacao(EstadoNivel estado)	—	Retorna recomendação padrão.
public	String getDescricaoFaixas()	—	Retorna o resumo das faixas.

As faixas são demonstrativas e não devem ser tratadas como valores oficiais.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

classificacao — interface e estratégias

`class ClassificadorPersonalizado` *implements* *ClassificadorNivel*

Implementação que permite definir faixas diferentes para cada estação.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	double	limiteSeca	Limite superior de seca.
private final	double	limiteAtencao	Início da faixa de atenção.
private final	double	limiteCheia	Início da faixa de cheia.
private final	double	limiteEmergencia	Início da emergência.

CONSTRUTORES

Assinatura	Descrição
<code>ClassificadorPersonalizado(double limiteSeca, double limiteAtencao, double limiteCheia, double limiteEmergencia)</code>	Valida se os limites são crescentes e armazena as faixas.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	<code>EstadoNivel classificar(double nivel)</code>	<code>NivelInvalidoException</code>	Classifica conforme as faixas do objeto.
public	<code>String gerarRecomendacao(EstadoNivel estado)</code>	—	Gera recomendação personalizada.
public	<code>String getDescricaoFaixas()</code>	—	Descreve os limites configurados.
public	<code>double getLimiteSeca()</code>	—	Retorna limite de seca.
public	<code>double getLimiteAtencao()</code>	—	Retorna limite de atenção.
public	<code>double getLimiteCheia()</code>	—	Retorna limite de cheia.
public	<code>double getLimiteEmergencia()</code>	—	Retorna limite de emergência.

Polimorfismo em tempo de execução

`EstacaoMonitoramento` armazena uma referência do tipo `ClassificadorNivel`. Essa referência pode apontar para `ClassificadorPadrao` ou `ClassificadorPersonalizado`. Assim, a chamada `classificar(nivel)` executa o comportamento do objeto associado a cada estação, sem alterar o restante do sistema.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

exception — exceções de negócio

class NivelInvalidoException *extends Exception*

Exceção verificada lançada quando o nível informado é negativo.

CONSTRUTORES

Assinatura	Descrição
NivelInvalidoException(String mensagem)	Encaminha a mensagem para a superclasse Exception.

class MedicaoDuplicadaException *extends Exception*

Exceção verificada lançada ao registrar duas medições na mesma data para a mesma estação.

CONSTRUTORES

Assinatura	Descrição
MedicaoDuplicadaException(String mensagem)	Encaminha a mensagem para a superclasse Exception.

class RegistroNaoEncontradoException *extends Exception*

Exceção verificada usada quando um ID não existe ou quando faltam dados para uma operação.

CONSTRUTORES

Assinatura	Descrição
RegistroNaoEncontradoException(String mensagem)	Encaminha a mensagem para a superclasse Exception.

Regras críticas protegidas

- O nível do rio não pode ser negativo.
- Uma estação não pode possuir duas medições na mesma data.
- Operações de atualização, remoção e consulta exigem registros existentes.
- Datas futuras, chuva negativa e temperaturas fora do intervalo aceito são rejeitadas por validação.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

service — regras de negócio

class EstacaoService

Camada responsável pelo CRUD e pela consulta das estações.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	Map<Integer, EstacaoMonitoramento>	estacoes	LinkedHashMap com estações por ID.
private	int	proximold	Gerador sequencial de IDs.

CONSTRUTORES

Assinatura	Descrição
EstacaoService()	Inicializa o mapa e o primeiro ID.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	EstacaoMonitoramento cadastrar(String nome, String cidade, String localizacao, ClassificadorNivel classificador)	IllegalArgumentException	Valida, cria e armazena uma estação.
public	List<EstacaoMonitoramento> listar()	—	Retorna cópia da lista de estações.
public	EstacaoMonitoramento buscarPorId(int id)	RegistroNaoEncontradoException	Localiza uma estação pelo ID.
public	EstacaoMonitoramento atualizar(int id, String nome, String cidade, String localizacao, ClassificadorNivel classificador)	RegistroNaoEncontradoException	Atualiza dados e, opcionalmente, o classificador.
public	EstacaoMonitoramento remover(int id)	RegistroNaoEncontradoException	Remove e retorna a estação.
public	int quantidade()	—	Retorna a quantidade cadastrada.

Coleção utilizada

LinkedHashMap mantém o acesso por ID e preserva a ordem de inserção durante a listagem.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

service — regras de negócio

class MedicaoService

Gerencia o CRUD de medições, as validações, os alertas e os cálculos de variação.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	EstacaoService	estacaoService	Serviço usado para localizar estações.
private final	List<AlertaHidrologico>	alertas	Alertas gerados em memória.
private	int	proximoldMedicao	Gerador de ID de medição.
private	int	proximoldAlerta	Gerador de ID de alerta.

CONSTRUTORES

Assinatura	Descrição
MedicaoService(EstacaoService estacaoService)	Recebe o serviço de estações e inicializa alertas e IDs.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	Medicao cadastrar(int estacaold, LocalDate data, double nivel, double chuva, double temperatura, String observacao)	RegistroNaoEncontradoException; NivelInvalidoException; MedicaoDuplicadaException	Valida, registra e pode gerar alerta.
public	List<Medicao> listarPorEstacao(int estacaold)	RegistroNaoEncontradoException	Retorna as medições ordenadas por data.
public	Medicao atualizar(int medicaold, LocalDate data, double nivel, double chuva, double temperatura, String observacao)	RegistroNaoEncontradoException; NivelInvalidoException; MedicaoDuplicadaException	Atualiza dados e reavalia alerta.
public	Medicao remover(int medicaold)	RegistroNaoEncontradoException	Remove uma medição localizada pelo ID.
public	Medicao buscarPorId(int medicaold)	RegistroNaoEncontradoException	Busca uma medição em todas as estações.
public	Medicao obterUltimaMedicao(int estacaold)	RegistroNaoEncontradoException	Retorna a medição mais recente.
public	double calcularVariacaoMaisRecente(int estacaold)	RegistroNaoEncontradoException	Subtrai o nível anterior do nível atual.
public	List<AlertaHidrologico> listarAlertas()	—	Retorna cópia dos alertas.
public	int quantidadeTotalMedicoes()	—	Soma as medições de todas as estações.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

service — relatórios

class RelatorioService

Responsável por gerar o arquivo CSV do histórico.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private static final	DateTimeFormatter	FORMATO_DATA	Formata datas como dd/MM/yyyy.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	Path exportarCSV(List<EstacaoMonitoramento> estacoes)	IOException; NivelInvalidoException	Cria relatorios/historico_monitora_rione gro.csv.

util — entrada de dados

class LeitorConsole

Centraliza leitura, validação e conversão de valores digitados no terminal.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private static final	DateTimeFormatter	FORMATO_DATA	Formato brasileiro de data.
private final	Scanner	scanner	Leitor da entrada padrão.

CONSTRUTORES

Assinatura	Descrição
LeitorConsole()	Inicializa o Scanner com System.in.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public	String lerTextoObrigatorio(String mensagem)	—	Repete até receber texto não vazio.
public	String lerTextoOpcional(String mensagem)	—	Lê texto que pode ficar vazio.
public	int lerInt(String mensagem)	—	Valida número inteiro.
public	int lerIntMinimo(String mensagem, int minimo)	—	Exige inteiro igual ou maior ao mínimo.
public	double lerDouble(String mensagem)	—	Aceita decimal com vírgula ou ponto.
public	LocalDate lerData(String mensagem)	—	Valida data no formato dd/MM/aaaa.
public	boolean lerSimNao(String mensagem)	—	Converte S/N para boolean.
public	void pausar()	—	Aguarda ENTER.
public	void fechar()	—	Fecha o Scanner.

Projeto 4 — MonitoraRioNegro — Monitoramento de Níveis de Cheia e Seca

main — aplicação e menu

CLASS Main

Ponto de entrada da aplicação e controlador dos menus do console.

ATRIBUTOS

Modificador	Tipo	Nome	Descrição
private final	LeitorConsole	leitor	Entrada de dados.
private final	EstacaoService	estacaoService	CRUD de estações.
private final	MedicaoService	medicaoService	CRUD de medições e alertas.
private final	RelatorioService	relatorioService	Exportação CSV.

CONSTRUTORES

Assinatura	Descrição
Main()	Instancia os componentes utilizados pela aplicação.

MÉTODOS

Modif.	Assinatura	Lança	Descrição
public static	void main(String[] args)	—	Cria Main e inicia a execução.
private	void executar()	Exceções tratadas no menu	Mantém o laço principal até a opção 0.

Menu principal

- 1 — Gerenciar estações; 2 — Gerenciar medições.
- 3 — Consultar situação atual; 4 — Exibir histórico e variação.
- 5 — Comparar duas ou mais estações; 6 — Exibir alertas.
- 7 — Exibir resumo estatístico; 8 — Exportar CSV.
- 9 — Adicionar dados demonstrativos; 0 — Encerrar.

Estrutura de pacotes

Pacote	Responsabilidade	Arquivos
model	Entidades e enum do domínio.	4
classificacao	Interface e estratégias polimórficas.	3
service	CRUD, cálculos, alertas e relatório.	3
exception	Exceções próprias de negócio.	3
util	Leitura e validação do console.	1
main	Inicialização e navegação dos menus.	1

Execução

Compilação: javac -encoding UTF-8 -d bin [todos os arquivos .java da pasta src]

Execução: java -cp bin br.com.monitorarionegro.main.Main

Rubrica de Avaliação (AV3) — Aplicação no MonitoraRioNegro

Critério	Peso	Evidência no projeto
Domínio dos conceitos de POO e documentação da API	3,0	Encapsulamento, interface, polimorfismo, enum, composição e exceções.
Funcionamento correto do sistema (CRUD e regras de negócio)	3,0	CRUD de estações e medições, validações, histórico, alertas e comparação.
Organização do código e do repositório GitHub	2,0	15 arquivos Java em seis pacotes, README.md, .gitignore e scripts de execução.
Qualidade dos slides e clareza da exposição oral	1,5	Apresentação visual com fluxo, classes, CRUD, POO e demonstração do console.
Contextualização amazônica coerente ao tema	0,5	Monitoramento dos ciclos de cheia e seca do Rio Negro e seus impactos regionais.

Síntese de conformidade

- Código-fonte compilável e organizado em pacotes.
- Quinze arquivos Java e mais de quatro classes próprias.
- Polimorfismo demonstrado por classificadores diferentes em tempo de execução.
- Coleções Java utilizadas para estações, medições, alertas, seleção e estatísticas.
- Menu funcional com operações CRUD e tratamento de exceções críticas.
- README.md com pré-requisitos, comandos de execução e orientação para o GitHub.

Aviso final: dados de exemplo e faixas de referência são utilizados exclusivamente para demonstração acadêmica.